L7-8 讲义：记忆系统与工具编排

课程: MBA 大模型智能体课程
课时: 第 7-8 讲 (共 3 小时)
适用: 讲师备课 / 学生预习与复习

本讲关键问题

作为 MBA 学生，学完这一讲你应该能回答：

1. **为什么 AI 助手用久了就"变傻"？** → 记忆系统的必要性
2. **如何让 AI"记住"用户偏好和历史？** → 三层记忆架构
3. **Context 窗口那么大，为什么还不够用？** → Context Engineering
4. **AI 怎么调用外部工具（数据库、API）？** → 工具设计原则
5. **AI 调用工具的风险是什么？如何防范？** → 安全与治理

为什么这节课很重要

前面我们学了 Agent 的基本概念和架构。但你可能会遇到一个问题：

场景：你搭建了一个客服 Agent，第一周效果很好。但两周后：- 用户问："我上次问的那个问题怎么样了？"→ Agent 一脸懵 - 用户说："按我之前说的偏好来"→ Agent 不记得偏好是什么 - 用户投诉："这个问题我已经问过三次了！"→ Agent 每次都当新问题

根本原因：Agent 没有"记忆"，它是"金鱼脑"——每次对话都是从零开始。

这节课解决什么：- 如何给 Agent 装上"记忆"- 如何管理有限的 Context 窗口 - 如何让 Agent 正确使用外部工具 - 如何防止 Agent"乱来"造成损失

目录

1. 为什么需要记忆？
2. 记忆系统三层架构
3. Context Engineering
4. 向量检索与 RAG
5. 长时间运行 Agent 的 Harness 设计 ☐ 新增
6. 工具设计与编排
7. 安全与治理
8. 实践案例
9. 参考资料

---

1. 为什么需要记忆？

1.1 一个真实的商业问题

为什么很多企业的 Agent 项目"上线即巅峰"？

我们观察到一个普遍现象：

时间	用户体验	原因
第 1 周	☐ ☐ ☐ ☐ ☐ 惊艳	新鲜感，简单问题为主
第 2 周	☐ ☐ ☐ 还行	开始问复杂问题，发现局限
第 4 周	☐ ☐ 失望	“它不记得我说过什么”
第 8 周	☐ 弃用	“还不如直接用 ChatGPT”

核心问题：- **记忆缺失**：不记得历史对话和用户偏好 - **工具不稳定**：有时能用，有时报错 - **偶发幻觉**：偶尔”胡说八道”，信任崩塌

1.2 LLM 的先天局限—MBA 必须理解

局限	技术原因	商业影响
上下文有限	最多 128K-200K tokens	长文档/长对话处理不了
无状态	每次 API 调用独立	不记得之前聊过什么
知识过时	训练数据有截止日期	不知道最近发生的事
无个性化	对所有用户一视同仁	无法学习用户偏好

关键洞察：LLM 本身是”无状态”的——你每次调用它，它都是”新生儿”，不记得之前的任何对话。所谓的”多轮对话”，其实是应用层把历史消息拼接到每次请求中。

1.3 记忆的商业价值—ROI 视角

场景	无记忆的损失	有记忆的收益	ROI 估算
智能客服	重复问答，平均处理时间 +35%	首问解决率提升 20%	人力成本降低 30%
销售助手	不了解客户，转化率低	个性化推荐，转化率 +15%	销售额提升
投研分析	每次从零开始，效率低	知识可复用，报告质量稳定	分析师效率翻倍
内部助理	员工体验差，弃用	跨会话连续协作	采纳率从 30%→80%

---

2. 记忆系统三层架构—像人脑一样设计

2.1 人类记忆 vs Agent 记忆

- ☐ 来源: Lilian Weng - Agent Memory

人类记忆	Agent 记忆	实现方式
感觉记忆	原始输入	Embedding 表示
短期记忆	上下文窗口	In-context Learning
长期记忆	外部存储	向量数据库

2.2 三层次设计

层次	内容	持续时间	技术实现
感知缓存	当前输入、工具返回、即时状态	秒级	Prompt 拼接、状态机

层次	内容	持续时间	技术实现
工作记忆	最近 N 轮对话 + 当前任务计划	会话级	Buffer/Summary Memory
长期记忆	用户画像、事实库、经验模式	跨会话	Vector DB + KV + 图数据库

结论：不是"选一种记忆"，而是"**分层组合**"。

2.3 记忆类型

类型	内容	存储方式	示例
情景记忆	具体事件和经历	向量 + 时间戳	"用户上周问过报销流程"
语义记忆	事实和知识	知识图谱/向量	"公司报销上限是 5000 元"
程序记忆	技能和操作步骤	结构化存储	"查询订单的 API 调用方式"

---

3. Context Engineering

- ☐ 来源: Anthropic - Context Window Engineering for Agents

3.1 什么是 Context Engineering

Context Engineering = 如何高效利用有限的上下文窗口

核心挑战：上下文窗口是稀缺资源，需要精心管理

3.2 Context Rot（上下文腐烂）

随着对话进行，上下文中累积的问题：

问题	表现	影响
过时信息	早期对话中的状态已改变	决策基于错误前提
冗余信息	重复的内容占用空间	减少有效信息量
噪声累积	无关信息越来越多	分散模型注意力
格式退化	长对话后格式变乱	输出质量下降

3.3 Context Compaction（上下文压缩）

```
原始对话（100K tokens）
    ↓ 摘要提取
关键信息（10K tokens）
    ↓ 结构化
精简上下文（5K tokens）
```

压缩策略：1. **滑动窗口**: 只保留最近 N 轮对话 2. **重要性采样**: 根据相关性筛选 3. **摘要替换**: 用摘要替换详细历史 4. **分层压缩**: 越早的信息压缩程度越高

3.4 Note-taking（笔记系统）

让 Agent 主动记录关键信息：

```
Agent内部笔记：
- 用户目标：完成Q4预算分析
- 已完成：收集了销售数据
- 待办：需要获取成本数据
- 约束：用户偏好表格形式
```

---

4. 向量检索与 RAG

4.1 为什么需要 RAG

RAG = Retrieval-Augmented Generation

```
传统LLM：问题 → LLM(仅用训练知识) → 回答

RAG：    问题 → 检索相关文档 → LLM(问题+文档) → 回答
```

4.2 Contextual Retrieval

> ☐ 来源: Anthropic - Contextual Retrieval

传统 RAG 的问题：切分后的 chunk 丢失上下文

解决方案：为每个 chunk 添加上下文说明

```
原始chunk："公司收入增长了20%"

上下文化chunk："[来自2024年Q3财报，讨论营收表现部分] 公司收入增长了20%"
```

效果：检索失败率从 5.7% 降至 1.9%

4.3 向量数据库选型

数据库	特点	适用场景
Pinecone	托管服务，易用	快速启动
Weaviate	开源，支持混合搜索	企业自建
Chroma	轻量级，嵌入式	原型开发
Milvus	高性能，分布式	大规模部署
pgvector	PostgreSQL 扩展	已有 PG 的团队

---

5. 长时间运行 Agent 的 Harness 设计

> ☐ 来源: Anthropic - Effective Harnesses for Long-Running Agents ☐ 新内容

5.1 核心挑战

长时间运行 Agent 的根本问题：**必须在离散的会话中工作，每个新会话开始时没有之前的记忆。**

> 想象一个轮班制的软件项目，每个新工程师到岗时完全不记得上一班发生了什么。

5.2 两部分解决方案

组件	职责	关键产出
Initializer Agent	首次运行时设置环境	init.sh、claude-progress.txt、初始 git commit
Coding Agent	每次会话增量推进	完成一个功能、更新进度、提交代码

5.3 关键文件设计

feature_list.json（功能列表）

```
{
  "category": "functional",
  "description": "New chat button creates a fresh conversation",
  "steps": [
    "Navigate to main interface",
    "Click the 'New Chat' button",
    "Verify a new conversation is created"
  ],
  "passes": false
}
```

设计要点：- 使用 JSON 格式（模型不太会错误修改 JSON）- 只允许修改 `passes` 字段 - 强制指令："不允许删除或编辑测试用例"

claude-progress.txt（进度日志） 记录每个会话做了什么，供下一个会话快速了解上下文。

5.4 典型会话启动流程

```
1. pwd → 确认工作目录
2. 读取 git logs + progress files → 了解近期工作
3. 读取 feature_list.json → 选择最高优先级未完成功能
4. 运行 init.sh → 启动开发服务器
5. 基础功能测试 → 确认系统正常
6. 开始实现新功能
```

5.5 失败模式与解决方案

问题	表现	解决方案
一次做太多	尝试一次性完成整个应用	强制增量进度（一次只做一个功能）
过早宣布完成	看到有进展就说"完成了"	使用 feature_list 强制测试验证
代码留下 Bug	功能实现一半就中断	每次结束前 commit 到干净状态
标记功能完成但未测试	代码写了但没端到端验证	强制使用浏览器自动化工具测试

5.6 增量进度的关键洞察

```
  错误方式：一次性构建完整应用
  正确方式：每次会话只完成一个功能

Session 1: 基础框架 + 功能1 → commit
Session 2: 读取进度 + 功能2 → commit
Session 3: 读取进度 + 功能3 → commit
...
```

好处：- 每次开始都有干净的代码状态 - 失败后容易回滚（git revert）- 减少"猜测上一次做了什么"的时间

5.7 测试验证的重要性

Agent 常见问题：认为写完代码就等于功能完成

解决方案：提供端到端测试工具（如 Puppeteer MCP），要求 Agent 以用户视角验证

```
Agent验证流程：
1. 启动开发服务器
2. 打开浏览器
3. 执行用户操作（点击、输入）
4. 验证结果正确
5. 只有验证通过才能标记为完成
```

---

6. 工具设计与编排

6.1 工具设计原则

原则	好的设计	坏的设计
单一职责	`search_web(query)`	`search_and_summarize(query)`
明确输入	详细参数描述	模糊的"data"参数
可预测输出	固定格式 JSON	随机格式文本
错误处理	返回结构化错误	抛出异常崩溃
幂等性	多次调用结果一致	依赖隐藏状态

6.2 工具 Schema 示例

```json
{
  "name": "query_database",
  "description": " 执行 SQL 查询获取业务数据，支持 SELECT 语句",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string",
        "description": "SQL SELECT 语句"
      },
      "database": {
        "type": "string",
        "enum": ["sales", "inventory", "users"],
        "description": " 目标数据库名称"
      },
      "limit": {
        "type": "integer",
        "default": 100,
        "description": " 返回结果数量上限"
      }
    },
    "required": ["query", "database"]
  }
}
```

6.3 工具网关 (Tool Gateway)

集中管理工具访问：

```
Agent请求 → 工具网关
                 认证授权
                 请求验证
                 速率限制
                 日志审计
                 路由到具体工具
```

---

7. 安全与治理

7.1 安全风险

风险类型	示例	危害级别
Prompt 注入	用户输入包含恶意指令	高
数据泄露	Agent 暴露敏感信息	高
越权操作	Agent 执行超出权限的动作	高
资源滥用	无限循环消耗资源	中

7.2 Guardrails 设计

```
用户输入 → [输入过滤] → LLM处理 → [输出过滤] → 工具执行 → [执行检查] → 返回
              ↓              ↓              ↓
           拦截恶意       检查合规       审批高危
```

7.3 权限分级

级别	操作类型	审批要求
L1	只读查询	自动通过
L2	低风险写入	自动 + 日志
L3	高风险操作	需人工确认
L4	关键操作	多人审批

---

8. 实践案例

8.1 企业知识库 Agent

架构:

```
用户问题
    ↓
意图识别
        知识查询 → RAG检索 → 生成回答
        数据查询 → SQL工具 → 格式化结果
        操作请求 → 权限检查 → 执行动作
```

8.2 长时间编码 Agent

基于 Anthropic 方案的实现:

```
项目初始化（Session 0）
  创建 feature_list.json（200+功能）
  创建 claude-progress.txt
  初始 git commit
  编写 init.sh

每日开发（Session 1-N）
  读取进度文件
  选择下一个功能
  实现 + 测试
  更新 passes: true
  git commit + 更新进度
```

---

9. 参考资料

核心阅读 ☐

- Anthropic: Context Window Engineering
- Anthropic: Contextual Retrieval
- Anthropic: Effective Harnesses for Long-Running Agents ☐ 新增
- Lilian Weng: Agent Memory

论文

- MemGPT - 虚拟内存管理
- Toolformer - 自学工具使用

实践指南

- OpenAI Function Calling Guide
- LangChain RAG Tutorial

安全

- OWASP LLM Top 10

课后思考题

1. 设计一个三层记忆系统来管理你们公司的客户服务历史。
2. 如果上下文窗口只有 8K tokens，你会如何设计压缩策略？
3. 为一个需要连续工作 3 天的 Agent 设计进度追踪方案。
4. 列出你业务中可能需要的 5 个工具，并定义它们的权限级别。

本讲义最后更新：2026-02-28